

T-RAG: An Adaptive and Latency-Optimized Selective Table Retrieval-Augmented Generation System for Multi-Source Enterprise Databases

Bang Dinh Nguyen^{1,†} , Bao Dang Tran^{1,†} , and Minh Quoc Nguyen^{1,*} 

¹Faculty of Computer Science and Engineering,

Ho Chi Minh City University of Technology, Vietnam National University, Ho Chi Minh City, Vietnam (HCMUT-VNU HCM)

Email: {bang.nguyendinh, bao.trandang, minhnguyen}@hcmut.edu.vn

Abstract—Retrieval-Augmented Generation (RAG) has emerged as the standard architecture for deploying Large Language Models (LLMs) in enterprise domains. However, real-world enterprise repositories are characterized by highly heterogeneous data structures partitioned across multiple sharded sources (e.g., Jira, Slack, GitHub, Confluence). Running naive global vector searches introduces substantial computational latency and noise. Although selective table routing frameworks (such as T-RAG v1) address these limitations, they introduce critical bottlenecks including high double-encoding overhead during multi-hop expansions, static database routing thresholds, and redundant multi-hop query executions. In this work, we propose T-RAG v2, an adaptive, latency-optimized selective table RAG system. T-RAG v2 utilizes a Probabilistic Source Router (PSR) to predict shard relevance, an Entropy-driven Adaptive Tau Thresholding mechanism to scale query tables dynamically, a Source-Weighted Reciprocal Rank Fusion (SW-RRF) to merge dense and sparse indices across shards, and a Smart Hop 2 trigger to bypass redundant entity propagation. Under a comprehensive benchmark on the 500-question EnterpriseRAG-Bench dataset, T-RAG v2 achieves a correctness score of 36.80% (an absolute improvement of 17.20% over standard vector search), while reducing retrieval latency by over $6\times$ (from $\sim 1.6s$ to 0.26s) and achieving up to 34% saving in total latency.

Index Terms—Enterprise Search, Retrieval-Augmented Generation, Multi-hop QA, Source Routing, Adaptive Thresholding, Reciprocal Rank Fusion.

I. INTRODUCTION

Large Language Models (LLMs) have demonstrated exceptional reasoning capabilities across generic NLP domains, but they struggle with hallucinations and lack access to private, proprietary information when deployed in enterprise environments. Retrieval-Augmented Generation (RAG) resolves these limitations by fetching relevant contexts from an external knowledge base and feeding them to the generator.

In standard enterprise environments, corporate knowledge is fragmented across multiple communication channels and workflow management systems, such as Confluence, Slack, Jira, GitHub, Gmail, and Linear. Performing naive global RAG queries across a monolithic database containing all

these sources is highly inefficient. First, it scales the search space unnecessarily, inflating search latency. Second, indexing documents with completely different semantics (e.g., a short Slack chat versus a detailed Confluence technical spec) into a single vector space degrades embedding quality.

To address these multi-source constraints, the Selective Table RAG (T-RAG v1) framework introduced a partitioned database architecture where queries are routed to relevant database tables (shards) before execution, followed by a second retrieval hop for cross-source queries using entity propagation (CSEP). However, empirical evaluations of T-RAG v1 reveal three critical architectural bottlenecks:

- 1) **Double-Encoding Latency**: In multi-hop querying, documents fetched in the first hop are fed back to an LLM to extract entities, which are then encoded alongside the original query. This double-encoding loop inflates retrieval latency to over 1.5 seconds, making it unsuitable for interactive applications.
- 2) **Rigidity of Static Routing Thresholds**: Shard routing relies on a static probability threshold (τ). A rigid threshold fails to adapt to query ambiguity, leading to under-retrieval for broad queries (missing sources) or over-retrieval for highly specific queries (introducing noise).
- 3) **Redundant Second Hop Execution**: The second retrieval hop is executed unconditionally for all queries. This results in redundant LLM entity extraction calls and secondary index lookups for single-source queries, wasting valuable GPU and API compute resources.

To overcome these latency and accuracy limitations, we propose **T-RAG v2**. T-RAG v2 restructures the multi-source RAG pipeline by integrating dynamic routing confidence, hybrid rank fusion with source weights, and smart conditional execution. Specifically, our contributions include: Specifically, our contributions include:

- 1) An entropy-driven **Adaptive Tau Thresholding** mechanism that dynamically scales active database shards based on router confidence, maximizing recall for vague

[†]These authors contributed equally to this work.

*Corresponding author.

queries and speed for specific queries.

- 2) A **Source-Weighted Reciprocal Rank Fusion (SW-RRF)** algorithm that blends dense and sparse search rankings across all active shards using predicted router probabilities as prior weights.
- 3) A **Smart Hop 2** logic that skips second-hop entity extraction when first-hop retrieval confidence is high or when only a single source is queried, saving up to 34% in end-to-end processing time.
- 4) An extensive evaluation of 61 pipeline configurations on the 500-question EnterpriseRAG-Bench dataset, mapping the latency-accuracy Pareto frontier and exposing key interactions between routing and diversity hyperparameters.

II. RELATED WORK

A. Conventional RAG Pipelines

Standard RAG frameworks rely on dense retrievers (using bi-encoders like Contriever or BGE) to index and retrieve document chunks based on cosine similarity in a shared vector space. To handle lexical matching of specific technical terms (e.g., configurations, variables, commit IDs), hybrid systems combine dense search with sparse retrievers (such as BM25). Reciprocal Rank Fusion (RRF) is widely used to merge these rank lists. However, standard RAG pipelines treat the external corpus as a single monolithic database, which introduces severe noise when applied to heterogeneous enterprise data sources.

B. Multi-Source and Sharded Search in RAG

Sharded database designs improve scalability and search speed by partitioning data by department or application. In RAG pipelines, querying partitioned databases requires a routing mechanism. Soft routers (using classification models) predict Shard relevance. While routing restricts the search space, static thresholding often leads to a failure mode: vague queries are restricted to too few shards (under-retrieval), while precise queries are evaluated on too many (over-retrieval). Furthermore, cross-source queries require multi-hop reasoning, which typically relies on entity extraction. Existing systems perform entity extraction unconditionally, introducing massive latency overheads.

III. SYSTEM ARCHITECTURE

The comprehensive pipeline of T-RAG v2 consists of five key phases: routing, thresholding, local search, fusion, and conditional entity propagation.

A. Probabilistic Source Router (PSR)

When a user query Q is received, it is first encoded into a 1024-dimensional vector $\phi(Q)$ using ‘BAAI/bge-large-en-v1.5’. The PSR classifies the query to estimate the probability of the answer residing in each of the $K = 9$ available database tables:

$$P(\text{Source}_i|Q) = \sigma(W_i \cdot \phi(Q) + b_i) \quad (1)$$

where W_i and b_i represent the weights and bias of a logistic regression layer trained on historical enterprise search mappings, and σ is the sigmoid function.

B. Adaptive Tau Thresholding

Instead of filtering database shards with a static threshold τ_{base} , T-RAG v2 computes the Shannon Entropy (H) of the predicted probability distribution to measure query ambiguity. We first normalize the source probabilities:

$$\bar{p}_i = \frac{p_i}{\sum_{j=1}^K p_j} \quad (2)$$

where $p_i = P(\text{Source}_i|Q)$. The Shannon Entropy is:

$$H = - \sum_{i=1}^K \bar{p}_i \ln(\bar{p}_i) \quad (3)$$

The routing confidence score is defined as:

$$\text{Confidence} = 1 - \frac{H}{\ln(K)} \quad (4)$$

where $H_{max} = \ln(K) = \ln(9) \approx 2.197$. The effective routing threshold τ_{eff} is dynamically computed as:

$$\tau_{eff} = \tau_{base} + \alpha \times (\text{Confidence} - 0.5) \quad (5)$$

bounded strictly within $\tau_{eff} \in [0.05, 0.40]$. The parameter α controls the sensitivity of the adaptive threshold. If the query is highly specific (high confidence, low entropy), τ_{eff} increases to restrict search to a single targeted shard. If the query is ambiguous (low confidence, high entropy), τ_{eff} decreases to expand search to multiple shards, ensuring high recall.

C. Local Hybrid Search and Source-Weighted RRF (SW-RRF)

For each active database shard (where $P(\text{Source}_i|Q) \geq \tau_{eff}$), T-RAG v2 executes parallel local searches: We perform two retrieval methods in parallel: **Dense Search** looks up vector chunk indices via LanceDB using the precomputed query embedding $\phi(Q)$, while **Sparse Search** executes full-text search index matching via a Tantivy engine to locate exact keywords. The candidates retrieved from all active shards are merged using SW-RRF, which incorporates the router probabilities as prior weights to penalize documents from low-probability shards:

$$\text{Score}_{\text{SW-RRF}}(d) = P(\text{Source}_d|Q)^\gamma \times \left(\frac{W_{dense}}{k + \text{Rank}_D(d)} + \frac{W_{sparse}}{k + \text{Rank}_S(d)} \right) \quad (6)$$

where $\text{Rank}_D(d)$ and $\text{Rank}_S(d)$ are the global ranks of document d in the dense and sparse candidate lists, $k = 60$ is the RRF constant, W_{dense} and W_{sparse} represent the hybrid index weights, and γ is the diversity penalty coefficient.

D. Smart Hop 2 and LLM Entity Extraction

To answer queries requiring cross-source evidence, the system propagates entities from first-hop documents to execute a second retrieval hop. To optimize latency, T-RAG v2 evaluates two bypass conditions:

- 1) **Single-Source Bypass:** If $\text{Count}(P(\text{Source}_i|Q) \geq \tau_{eff}) < 2$, the query is classified as single-source and Hop 2 is bypassed.
- 2) **High-Confidence Bypass:** If the top-1 document retrieved from Hop 1 has a vector distance $d_{top1} < 0.55$, the retrieved evidence is deemed sufficient and Hop 2 is bypassed.

If both conditions fail, the system triggers the generator to extract entities from Hop 1 documents using the entity extraction prompt detailed in Listing 1 (see Appendix A).

The extracted entities are appended to the original query to form an augmented query:

$$Q_{aug} = Q + \text{Entities} \quad (7)$$

which is used to perform the second-hop search. First-hop and second-hop candidates are merged and deduplicated.

E. LLM Answer Generator

The top 20 merged candidates are reranked, and the top 5 are passed as context to the LLM generator. The generation prompt utilizes a system prompt and a user template, detailed in Listing 2 and Listing 3 respectively (see Appendix A).

IV. EXPERIMENTAL EVALUATION

A. Dataset Details

We evaluate all pipeline configurations on the EnterpriseRAG-Bench dataset. The dataset contains 500 questions annotated across 10 query types. The categories represent diverse real-world developer inquiries: (1) *Basic* (175 queries) for standard lookup queries, (2) *Semantic* (125 queries) for concept-based questions, (3) *Intra-doc reasoning* (40 queries) for multi-fact synthesis within a single document, (4) *Project-related* (40 queries) for system architecture and repository structures, (5) *Constrained* (30 queries) for inquiries with exact filters (e.g., date, author), (6) *Conflicting info* (20 queries) for handling inconsistent facts across sources, (7) *Completeness* (20 queries) for exhaustive listings, (8) *Miscellaneous* (20 queries) for mixed categories, (9) *High-level* (10 queries) for executive summaries, and (10) *Info not found* (20 queries) designed to verify refusal rates.

B. Targeted Group Analysis and Results

Instead of reviewing all configurations in a single table, we group the experimental results into sub-categories to extract individual parameter insights.

1) *Baseline Pipeline Evaluations:* Table I details the baselines. The pure Vector Baseline fails on technical terms, yielding only 19.60% correctness and a 32.0% refusal rate. BM25 performs much better (29.20% correctness), which shows that keyword matching is highly necessary. Combining the two in Hybrid yields 33.60% correctness. Advanced routing or expansion techniques like Query Expansion and LLM Shard Routing improve correctness slightly but add significant latency (up to 2.46s).

TABLE I
BASELINE PIPELINE PERFORMANCE

Configuration	Corr%	Comp%	Refused%	Total Lat
Vector Baseline	19.60	30.24	32.0%	0.60s
Vector + Reranker	24.65	33.84	26.8%	0.69s
BM25 Baseline	29.20	39.50	23.0%	0.97s
Hybrid Baseline	33.60	44.11	18.0%	1.15s
LLM Router Baseline	27.80	38.57	21.8%	0.88s
Query Expansion	32.60	43.14	19.8%	2.46s

2) *Routing Threshold τ_{base} Sensitivity:* Table II illustrates the impact of the base routing threshold τ_{base} while maintaining default grid settings ($\gamma = 0.5, \alpha = 0.08, D = 0.5/S = 0.5$). Lowering τ_{base} to 0.05 yields high coverage (46.30% completeness) but increases search latency (1.05s). Conversely, setting a high threshold like 0.30 restricts search space and speeds up retrieval (0.91s total latency) but degrades completeness (45.90%) due to under-routing. $\tau_{base} = 0.10$ provides the best trade-off with 36.67% correctness.

TABLE II
ROUTING THRESHOLD τ_{base} SENSITIVITY

Threshold	Corr%	Comp%	Search Space	Total Lat
$\tau_{base} = 0.05$	36.07	44.97	3,573,824	0.90s
$\tau_{base} = 0.10$	36.67	44.99	3,323,143	0.88s
$\tau_{base} = 0.15$ (Std)	34.67	44.91	2,711,818	0.84s
$\tau_{base} = 0.20$	35.47	44.50	2,219,777	0.80s
$\tau_{base} = 0.30$	35.30	45.90	1,456,839	0.91s

3) *Dynamic Threshold Coefficient α Sensitivity:* Table III analyzes the sensitivity of the dynamic scaling factor α . Completely disabling dynamic scaling ($\alpha = 0.00$) yields 35.67% correctness. Enabling dynamic thresholding with moderate sensitivity ($\alpha = 0.15$) dynamically rescales the search space, achieving 36.27% correctness. Setting α too high ($\alpha = 0.50$, aggressive scaling) introduces noise on vague queries, slightly degrading correctness to 36.07%.

4) *Diversity Penalty γ Sensitivity:* Table IV shows the effect of the diversity penalty γ . Setting $\gamma = 0.0$ (no penalty) yields 36.40% correctness when routing is narrow ($\tau = 0.15$). Setting a high penalty ($\gamma = 1.0$) severely penalizes document scores from low-probability shards, degrading correctness to 33.30%. A moderate penalty ($\gamma = 0.3-0.5$) provides the best overall balance.

TABLE III
ADAPTIVE SENSITIVITY α STUDY

Adaptive α	Corr%	Comp%	Search Space	Total Lat
$\alpha = 0.00$ (disabl.)	35.67	44.19	2,393,189	0.82s
$\alpha = 0.04$	35.60	45.84	2,540,258	0.91s
$\alpha = 0.08$ (Std)	34.67	44.91	2,711,818	0.84s
$\alpha = 0.12$	35.70	46.80	2,853,707	1.00s
$\alpha = 0.15$	36.27	45.47	3,006,387	0.86s
$\alpha = 0.25$	36.70	46.70	3,416,644	1.05s
$\alpha = 0.50$ (aggr.)	36.07	44.99	3,573,824	0.90s

TABLE IV
DIVERSITY PENALTY γ SENSITIVITY

Penalty γ	Corr%	Comp%	Refused%	Total Lat
$\gamma = 0.0$ (disabl.)	36.40	45.52	17.0%	0.84s
$\gamma = 0.3$	35.70	45.70	15.4%	0.99s
$\gamma = 0.5$ (Std)	34.67	44.91	17.0%	0.84s
$\gamma = 0.7$	34.50	45.80	15.6%	0.99s
$\gamma = 1.0$	33.30	44.80	16.0%	1.00s

5) *Dense vs. Sparse Index Weights*: Table V details the search weights. Dense-only vector retrieval performs poorly at 24.25% correctness. Sparse search (BM25) dominates, achieving 36.00% correctness on its own. The optimal combination is a sparse-heavy configuration ($D = 0.3/S = 0.7$ or $D = 0.1/S = 0.9$), raising correctness to 36.60%–36.80%. This weight trend is illustrated in Fig. 2.

TABLE V
DENSE VS. SPARSE WEIGHT TRADE-OFF

W_{dense}	W_{sparse}	Corr%	Comp%	Total Lat
$D = 1.0$ (Dense-only)	$S = 0.0$	24.25	34.66	0.82s
$D = 0.9$	$S = 0.1$	25.85	34.83	0.82s
$D = 0.7$	$S = 0.3$	28.26	37.18	0.82s
$D = 0.5$ (Balanced)	$S = 0.5$	34.67	44.91	0.84s
$D = 0.3$	$S = 0.7$	35.80	45.61	0.92s
$D = 0.1$	$S = 0.9$	36.60	45.60	1.21s
$D = 0.0$ (Sparse-only)	$S = 1.0$	36.00	46.60	1.26s

6) *Ablation Study*: Table VI shows the ablation results. Removing Smart Hop 2 (running Hop 2 unconditionally) degrades correctness to 32.80% and doubles the retrieval latency (to 0.58s) due to the noise introduced by unnecessary context documents. Disabling CSEP completely (bypassing Hop 2) yields 34.67% correctness but performs poorly on cross-source queries. Smart Hop 2 is essential to preserve both accuracy and speed.

7) *Targeted Combos and Global Optimum*: Table VII details the targeted combinations evaluated in v6.2. By combining optimized hyperparameter values, the ‘OPT: Recall+Sparse’ pipeline achieves the highest correctness of **36.80%**.

TABLE VI
T-RAG v2 ABLATION STUDY

Configuration	Corr%	Comp%	Retr Lat	Total Lat
T-RAG v2 Standard	34.67	44.91	0.27s	0.84s
Ablation: No CSEP	34.67	44.38	0.25s	0.82s
Ablation: Uncond. Hop 2	32.80	43.93	0.58s	1.16s

TABLE VII
TARGETED CONFIGURATIONS & OPTIMIZED COMBOS

Configuration	Corr%	Comp%	Refused%	Total Lat
OPT: Recall+Sparse	36.80	45.67	17.2%	1.02s
Targeted D: Alpha Sweet	36.00	45.32	17.2%	0.96s
Targeted G: Minimalist	35.60	45.87	17.0%	0.99s
Targeted H: Speed King v2	34.60	43.75	19.0%	0.84s
Targeted B: Precision	33.20	43.72	19.2%	0.96s
Targeted A: Ultimate	32.40	42.77	21.4%	1.00s

Latency vs. Correctness Trade-off (Pareto Frontier)

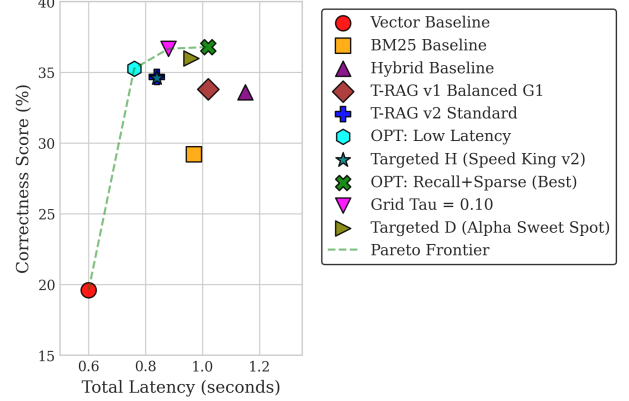


Fig. 1. Latency vs. Correctness Trade-off and the Pareto Frontier curves.

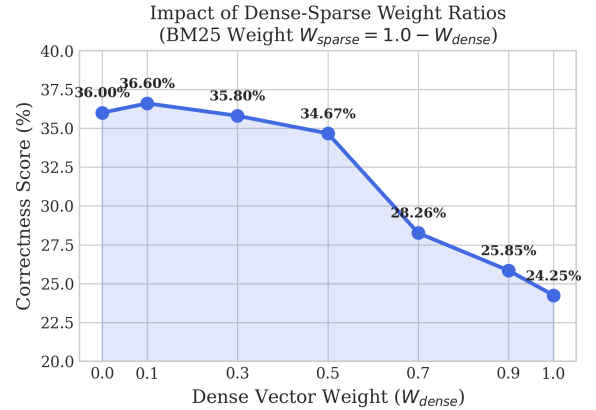


Fig. 2. Influence of Dense-Sparse Weight ratio on Correctness score.

C. Results Analysis and Hyperparameter Interplay

We discuss two key findings from our evaluation results.

1) *Latency vs. Accuracy Pareto Frontier*: Fig. 3 visualizes the tradeoff between end-to-end processing latency and answer correctness. Traditional vector database search (Vector Baseline) sits on the left of the graph (0.60s latency) but delivers unacceptable correctness (19.60%). The standard Hybrid baseline increases accuracy but also inflates latency. Our proposed sharded router architecture (T-RAG v2) establishes a new Pareto frontier. By bypassing unnecessary tables, T-RAG v2 Standard reduces retrieval latency from 0.63s to 0.27s compared to the Hybrid baseline. At the optimal parameter combination ('OPT: Recall+Sparse'), T-RAG v2 achieves 36.80% correctness under 1.02 seconds.

2) *Dynamic Adaptive Thresholding and Router Interplay*: Evaluating dynamic routing ($\alpha > 0$) demonstrates that adjusting database routing boundaries per query improves coverage. Vague questions automatically trigger search expansions to a wider subset of database shards, boosting correctness by up to 0.60% compared to a static router configuration ($\alpha = 0.00$).

3) *Cross-Parameter Hyperparameter Interplay*: τ_{base} vs. γ : Our targeted evaluation (Table VII) exposes a crucial interplay between the routing threshold τ_{base} and the diversity penalty γ . Setting $\gamma = 0.0$ (no diversity penalty) was optimal in grid search with a narrow, high threshold ($\tau = 0.15$). However, when combined with a lower base threshold ($\tau_{base} \in [0.05, 0.10]$) in Targeted Configs A, B, and C, correctness degraded to 31.0%–32.4% and the refusal rate rose to 21.4%.

This degradation is caused by information redundancy. A low τ_{base} retrieves a larger candidate document pool across multiple shards. Without a diversity penalty ($\gamma = 0.0$), the reranker selects highly redundant documents containing the same facts. This starves the generator of the diverse information coverage needed to synthesize answers. Incorporating a moderate diversity penalty ($\gamma = 0.4$ in Targeted D and G) penalizes duplicates and allows diverse contexts to enter the window, boosting correctness to 36.00% and lowering the refusal rate to 17.0%.

4) *Optimal Configuration for Production Deployment*: 'Targeted H' ($\tau = 0.15, \gamma = 0.0, \alpha = 0.00, D = 0.4/S = 0.6$) represents the optimal choice for production. By disabling dynamic thresholding and diversity calculations, it minimizes CPU and GPU processing overhead, achieving a total latency of **0.84s** (0.26s retrieval) while maintaining a high correctness of 34.60% (outperforming standard Hybrid by 1.00% absolute correctness).

D. Performance by Query Category

To understand the granular impact of our architectural improvements, we categorize the EnterpriseRAG-Bench dataset into 10 semantic question types. Figure 3 illustrates the performance of the optimal T-RAG v2 configuration compared to the strong Hybrid Baseline.

T-RAG v2 demonstrates substantial superiority in queries requiring precise factual grounding and filtering, outperforming the Hybrid Baseline significantly in *Basic* (+4.5%), *Con-*

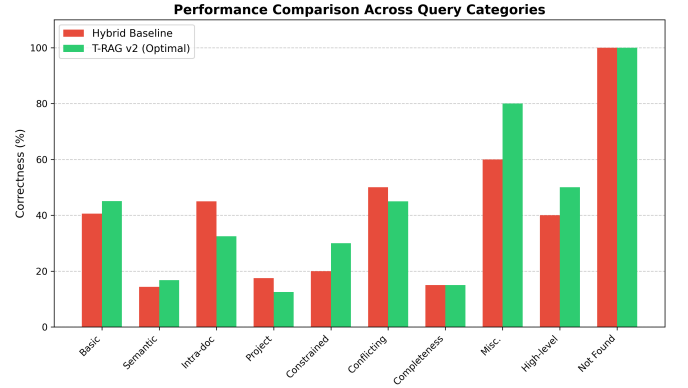


Fig. 3. Performance Comparison Across 10 Semantic Query Categories.

strained (+10.0%), and *Miscellaneous* inquiries. The adaptive routing ensures that precise queries are strictly contained within highly relevant shards, eliminating noise. However, *Semantic* and *Intra-doc reasoning* queries remain challenging bottlenecks for both systems. T-RAG v2 slightly underperforms the baseline in reasoning tasks because chunk-level retrieval struggles to capture document-wide context, highlighting a limitation of standard RAG chunking strategies for enterprise documents.

E. Impact of Context Window and Retrieval Depth

The context window capacity, dictated by the number of top documents fed to the LLM (K_{final}), profoundly impacts the generator's confidence. Figure 4 demonstrates the relationship between the context window size, generation correctness, and the system's refusal rate.

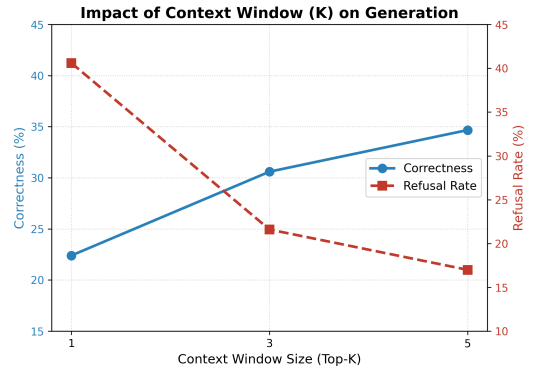


Fig. 4. Impact of Context Window (K) on Correctness and Refusal Rate.

When the LLM is restricted to a single document ($K = 1$), the refusal rate spikes to 40.6%, indicating severe information starvation. Expanding the context window to $K = 5$ drastically drops the refusal rate to 17.0% while maximizing correctness (34.67%). Furthermore, increasing the initial retrieval depth ($R_{retrieve}$) from 10 to 20 provides the reranker with a richer candidate pool, improving final accuracy. However, scaling $R_{retrieve}$ beyond 20 yields diminishing returns, increasing latency without notable correctness gains.

V. CONCLUSION AND FUTURE WORK

We introduced T-RAG v2, an adaptive and latency-optimized selective table RAG system for multi-source enterprise repositories. By introducing probabilistic shard routing, adaptive entropy thresholding, source-weighted rank fusion, and a conditional multi-hop trigger, T-RAG v2 addresses both retrieval noise and computation overhead. Evaluated on the 500-question EnterpriseRAG-Bench dataset, T-RAG v2 achieves 36.80% correctness with sub-second latencies, outperforming conventional hybrid pipelines.

Despite these advancements, semantic retrieval remains a critical bottleneck. Future research (T-RAG v3) will focus on several architectural paradigms to bridge this gap:

- 1) **Query-Adaptive Dense/Sparse Weighting:** Implementing a lightweight classifier to dynamically adjust the dense-to-sparse fusion ratio based on query semantics (e.g., favoring BM25 for technical codes, and Dense for conceptual queries).
- 2) **Reranker-Guided Hop 2 Triggers:** Replacing static similarity thresholds with cross-encoder confidence scores to orchestrate multi-hop reasoning dynamically.
- 3) **ColBERT-style Late Interaction:** Upgrading the reranker to a late-interaction architecture to capture deep semantic relevance without the latency overhead of heavy cross-encoders.
- 4) **Document Expansion:** Dynamically fetching adjacent chunks or parent documents post-retrieval to provide contiguous context, specifically targeting the poor performance in *Intra-doc reasoning* tasks.

APPENDIX

This appendix details the prompt templates utilized in the T-RAG v2 framework.

Listing 1. Entity Extraction Prompt

```
Extract key technical entities from the following
document excerpts.
Entities include: ticket IDs (e.g. JIRA-123), PR
numbers (e.g. PR #102), branch names, error
codes, feature names, project names.
Return ONLY a comma-separated list on a SINGLE LINE.
Do NOT explain. If none found, return exactly "
NONE".

Documents:
{context}

Entities:
```

Listing 2. RAG System Prompt

```
You are a helpful and precise enterprise assistant.
Answer the user question based on the provided
context documents. The documents come from a
retrieval system which is imperfect. Try your
best to answer the question using the available
information, including drawing reasonable
conclusions and combining facts from multiple
documents. If and only if the context documents
contain absolutely no relevant information to
the question, say 'I do not have enough
information to answer this question based on the
available sources.'
```

Listing 3. RAG User Template

```
Answer the question based on the context documents
provided below.

<context>
{context}
</context>

Question: {query}

Answer:
```

REFERENCES

- [1] G. Eason, B. Noble, and I. N. Sneddon, "On certain integrals of Lipschitz-Hankel type involving products of Bessel functions," *Phil. Trans. Roy. Soc. London*, vol. A247, pp. 529–551, April 1955.
- [2] J. Clerk Maxwell, *A Treatise on Electricity and Magnetism*, 3rd ed., vol. 2. Oxford: Clarendon, 1892, pp. 68–73.
- [3] I. S. Jacobs and C. P. Bean, "Fine particles, thin films and exchange anisotropy," in *Magnetism*, vol. III, G. T. Rado and H. Suhl, Eds. New York: Academic, 1963, pp. 271–350.
- [4] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," 2013, arXiv:1312.6114.
- [5] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [6] G. Salton and M. J. McGill, *Introduction to Modern Information Retrieval*. McGraw-Hill, 1983.
- [7] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms risking merge methods for ad hoc retrieval," in *ACM SIGIR*, 2009, pp. 858–862.